

Eine Diktier-App, die den Rechner nicht verlässt

Taste halten, sprechen, loslassen — der Text landet im Feld, in dem der Cursor steht. Oder ganz ohne Taste: Startwort sagen, sprechen, aufhören. Dazwischen liegen Spracherkennung, ein lokales Sprachmodell und sieben Schutzschichten, die verhindern, dass etwas eingefügt wird, das **niemand gesagt hat**. Kein Cloud-Dienst, kein Konto, keine Internetverbindung im Betrieb.

20.848

Zeilen Programm

931

Tests, grün

1.190

Diktate im Betrieb

1,5 %

Rückfall-Quote

01

Die Grundidee

Sprechen ist etwa dreimal so schnell wie Tippen. Der Haken: Gesprochenes *klingt* nicht wie geschriebener Text. Man setzt an, korrigiert sich, wiederholt, sagt „ähm“. Eine reine Spracherkennung liefert genau das — unbrauchbar für alles, was jemand lesen soll.

Fleech schiebt deshalb ein Sprachmodell dazwischen, das aufräumt. Und weil das Modell prinzipiell auch *erfinden* kann, steht dahinter eine Kette von Prüfungen, die jede Ausgabe gegen das gesprochene Original hält.

Die Leitfrage des ganzen Programms: **Steht am Ende das da, was gesagt wurde?** Im Zweifel wird der Rohtext eingefügt, nicht die schönere Fassung.

Was daraus folgt

LOKAL, NICHT AUS PRINZIP

Diktate enthalten Passwörter, Krankmeldungen, Gehälter. Cloud-Erkennung hieße, das an einen Dienst zu geben. Der Cloud-Pfad wurde mit 3.6.0 ersatzlos entfernt — es gibt keinen Schalter, der ihn zurückholt.

WORTGETREU VOR SCHÖN

Das Modell darf glätten, nicht umschreiben. Ein Guard vergleicht die Wortmenge vorher/nachher; wer mehr ergänzt als erlaubt, dessen Ausgabe wird verworfen.

02

Die Signalkette

Ein Diktat durchläuft immer dieselben Stufen. Die Zeiten sind Mittelwerte aus 1.190 echten Diktaten auf einer RTX 4070.

Mikrofon

16 kHz Mono, nur im RAM

Artefakt-Filter

Stille, Loopback

Spracherkennung

Whisper large-v3-turbo

▼ Modus-Routing

STANDARD

Bereinigung

gemma3:4b

SAFE-WORD

Befehl

ersetzen, löschen

FORMEL

Parser

ohne Modell

▼

Die sieben Guards

prüfen jede Ausgabe gegen das Gesprochene

BESTEHT 98,5 %

Text einfügen

Zwischenablage + Strg+V

FÄLLT DURCH 1,5 %

Rohtext einfügen

lieber unbereinigt als erfunden

Verlauf + Projekt-Gedächtnis

lokal, abschaltbar

Der wichtigste Zweig ist der rechte unten: Scheitert die Bereinigung, geht trotzdem Text raus — nur eben der unbereinigte.

01 Aufnahme Taste gehalten

16 kHz Mono, gepuffert im Arbeitsspeicher — nie auf die Festplatte. Bei Bedarf senkt Fleech die Lautstärke anderer Programme („Ducking“), damit Musik nicht ins Mikrofon läuft.

`fleech/audio.py` · `fleech/audiofocus.py`

02 Artefakt-Filter < 5 ms

Zu kurze Aufnahmen und reine Stille werden verworfen, bevor irgendetwas rechnet. Ein struktureller Loopback-Schutz erkennt, wenn Fleech seine eigene Ausgabe wieder hört.

`fleech/pipeline.py`

03 Spracherkennung 810 ms Ø

`faster-whisper large-v3-turbo` auf der Grafikkarte. Fehlt CUDA, fällt es automatisch auf den Prozessor zurück — langsamer, aber es läuft.

Vorab bekommt die Erkennung einen **Priming-Satz**: das persönliche Wörterbuch, Baustein-Kürzel, das Safe-Word und seit 5.1.0 das [gelernte Fachvokabular](#). Damit kommt „MCP-Server“ als MCP-Server an und nicht als „MCP Server“.

`fleech/stt/faster_whisper_stt.py`

04 Modus-Routing < 1 ms

Drei Wege: **Bereinigung** (der Normalfall), **Befehl** (ein gesprochenes Safe-Word macht die Äußerung zur Anweisung) oder **Formel** (gesprochene Mathematik wird zu LaTeX).

`fleech/routing.py`

05 Sprachmodell 4,1 s 0

`gemma3:4b` über Ollama, lokal. Ein adaptives Routing schickt triviale Diktate an einem großen Modelldurchlauf vorbei: 3 % laufen ganz ohne KI, 13 % über den schnellen Weg, 81 % über die volle Bereinigung.

Formeln entstehen in einem **deterministischen Parser**, nicht im Modell – bei Mathematik ist Raten keine Option.

`fleech/llm/client.py` · `fleech/formula.py`

06 Die sieben Guards < 10 ms

Jede Modellausgabe wird geprüft, bevor sie eingefügt wird. [Einzelnen beschrieben weiter unten](#). Fällt sie durch, kommt der Rohtext – gemessene Quote: 1,5 %.

`fleech/textutils.py` · `fleech/pipeline.py`

07 Einfügen ~50 ms

Über die Zwischenablage plus simuliertes Strg+V – der einzige Weg, der in jedem Windows-Programm funktioniert. Der vorherige Inhalt wird wiederhergestellt. Fleech prüft aktiv, ob die Zwischenablage wirklich gesetzt wurde, statt blind zu warten.

`fleech/injection.py` · `fleech/ui/focusrestore.py`

03

Die sieben Guards

Jeder einzelne entstand aus einem realen Vorfall. Sie sind der Grund, warum die Rückfall-Quote bei 1,5 % liegt statt bei null: Wenn Fleech unsicher ist, entscheidet es sich für den Rohtext.

GUARD	PRÜFT	ENTSTANDEN AUS
Wortgetreue	Wortmenge vorher / nachher	Das Modell formulierte ganze Sätze um, die so nie gesagt wurden.
Inflation	Ergänzungen am Satzende	Höfliche Schlussfloskeln, die niemand diktieren hatte.

GUARD	PRÜFT	ENTSTANDEN AUS
Wiederholungsfilter	≥ 4 gleiche Wörter am Ende	Whisper-Schleifen bei Stille. Schwelle 4, weil „wirklich wirklich sehr sehr sehr gut“ echt vorkam.
Fremdsprach-Tail	Fremde Diakritika, englische Füller	„Seekers Odoo Time Go Go Go and Let me and or“ — angehängt an ein deutsches Diktat.
Kauderwelsch	Vier Signale, Schnitt ab zwei	Wortsalat am Diktatende, den die anderen drei Filter durchließen.
Meta-Präambel	„Hier ist der bereinigte Text:“	Das Modell kommentierte seine eigene Arbeit.
Abschneide-Erkennung	Antwort am Kontextfenster abgebrochen	Lange Diktate endeten mitten im Satz. Dann: Rohtext statt halber Satz.

Warum `num_ctx` Pflicht ist: Ollama lädt Modelle immer mit 4096 Token Kontext, egal was das Modell könnte – und der OpenAI-Aufsatz ignoriert jede Option dagegen. Allein der Bereinigungs-Prompt belegt ~3000 Token. Fleech spricht deshalb bei lokalem Ollama dessen eigene `/api/chat` mit `options.num_ctx` an.

04

Profile und Apps

Ein Profil bestimmt, was aus dem Diktat wird. Dieselben gesprochenen Informationen werden je nach Profil zu einer Mail, zu einem KI-Prompt oder zu einer Stichpunktliste.

● STANDARD

Bereinigter Text, so wie gesprochen. Nur die Sprechweise fällt weg.

● E-MAIL

Anrede, Absätze, Grußformel, gehobenerer Ton. Der Absendername kommt als Nachrichten-Kontext, nicht in den System-Prompt.

● KI-PROMPT

Zieht Wiederholungen zusammen, dampft Herleitungen ein. Gemessen: 92 gesprochene Wörter → 70, ohne eine Anforderung zu verlieren.

● STICHPUNKTE

Jede genannte Sache ein Punkt. **Keine Zusammenfassung** — weggelassen wird nichts, nur die Sprechweise verschwindet.

Die Apps-Seite

Die Zuordnung steht dort, wo die Frage entsteht: bei der Anwendung. Drei Spalten — welche Programme es gibt, welches Profil dort automatisch gilt, und zwischen welchen der Profil-Hotkey dort überhaupt wechselt.

In einem KI-Chat sind das andere als in Word: Statt durch acht Profile zu tippen, wechselt man dort nur zwischen zweien. Feinere Regeln greifen auf den Fenstertitel, sodass derselbe Editor mit Code und mit Notizen verschieden behandelt wird.

Während einer laufenden Aufnahme gilt die App, in der *gestartet* wurde — auch wenn man zwischendurch wechselt. Dorthin kehrt der Cursor zurück, dort landet der Text.

05

Projekt-Gedächtnis NEU IN 5.1.0

Fleech lernt aus dem, was es einfügt, die Fachbegriffe eines Zusammenhangs und gibt sie beim nächsten Diktat an die Erkennung weiter. Das überlebt Neustarts.

Eingefügter Text

der bereinigte, nicht das Rohtranskript

Begriffe extrahieren

Binnenversalien, Ziffern, Punkte



kontext.db

je App und Titel-Segment

▲ beim nächsten Diktat

SPEZIFISCH

Fenster-Segmente

„Fleech“, „Analysis II“

GRUNDSTOCK

App-weit

was überall in dieser App gilt



Priming-Satz an Whisper

max. 25 Begriffe, 0,6 ms

Zwei Ebenen: Was im konkreten Fenster gilt, steht vorn; darunter der Bestand des ganzen Programms.

Was als Fachbegriff zählt

Drei harte Signale, an 1.189 echten Diktaten kalibriert:

Binnenversalien (`MCP-Server` , `PySide6`), eine Ziffer im Wort

(`gemma3` , `x_3`) oder ein Punkt im Wortinneren

(`share.finland.xyz`).

Ein Ansatz, der gemessen und verworfen wurde



Warum die Titel-Segmente und nicht der ganze Titel



Es geht kein Inhalt an die KI. Eine Projekt-Zusammenfassung im Prompt wäre mächtiger, würde aber jedes Diktat verlangsamen und dem Modell Material geben, aus dem es ergänzen kann – genau die Halluzinationen, gegen die die sieben Guards stehen. Vokabular kann nichts erfinden.

VORGANG	KOSTEN	WANN
Begriffe abrufen	0,6 ms	vor der Erkennung
Lernen	3,1 ms	nach dem Einfügen
Erstbefüllung	3,7 s	einmalig, im Hintergrund

Was seit 5.1.0 dazukam

FREIHAND — OHNE TASTE · 5.3.0

Startwort sagen, sprechen, aufhören. Ein Gate aus zwei Stufen: Das billige VAD hört auf Sprache überhaupt, das kleine Modell prüft nur dann auf das Startwort.

Gemessen: **1,2 % Dauerlast** eines Kerns, im Leerlauf startet das Modell null Mal. Im Speicher liegen immer nur die letzten zwei Sekunden.

ENGLISCH · 5.4.0

Zwei der vier Kauderwelsch-Merkmale sind sprachgebunden — bei englischem Diktat waren beide dauerhaft gesetzt, und zwei bedeuten Schnitt. **Jedes englische Diktat wäre gekürzt worden.** Die Merkmale spiegeln sich jetzt mit der Sprache.

LIVE-VORSCHAU · 5.2.0

Der Rohtext steht nach ~0,8 s in der Pille, während das Modell noch vier Sekunden arbeitet. Die Statusmeldungen hängen an den echten Schritten, nicht an einem geschätzten Balken.

NACHTRÄGLICH ÄNDERN · 5.2.0

Rechtsklick im Verlauf → „Neu bereinigen als ...“. Setzt auf dem *Rohtranskript* auf und läuft durch dieselben Guards. Ergebnis in die Zwischenablage, nicht blind ins zuletzt benutzte Feld.

DIE OBERFLÄCHE IST GESCHICHTET · NACH 5.4.0

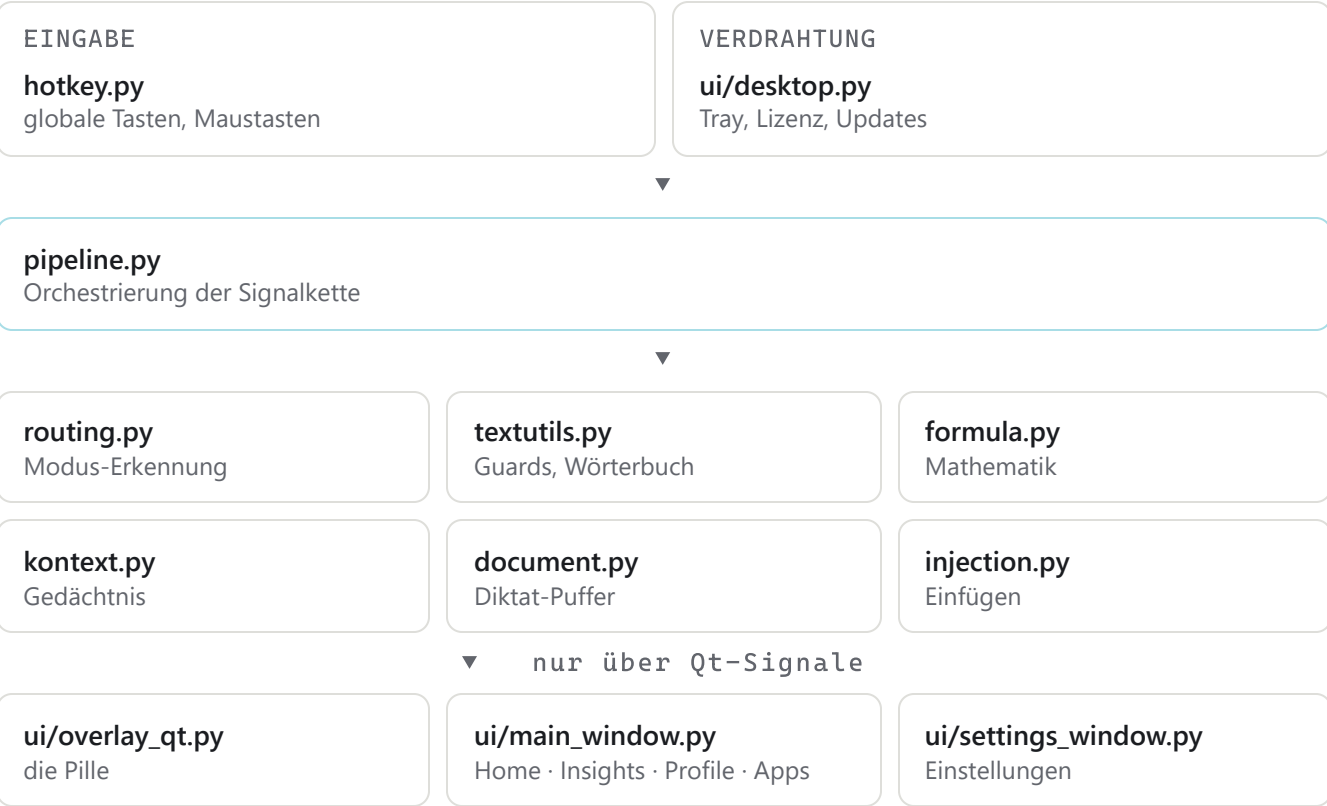
Vier Seiten, vier Dialoge und fünfzehn Widgets lagen in einer Datei mit 2.970 Zeilen. Schlimmer als die Länge war die Richtung: Sechs Module zogen ihre **Farben aus dem Hauptfenster** — der Update-Dialog importierte es, nur um eine Kartenfarbe zu kennen.

Heute: `theme` → `widgets` → `dialogs` → `pages/` → `main_window` → `desktop`. Ein Test hält fest, dass `theme` nichts aus der App importiert, keine Seite ihr Fenster kennt und keine Importzyklen entstehen.

Ein Prompt für Englisch war nötig, und das wurde gemessen: Mit dem deutschen Prompt hat das Modell englische Diktate **ins Deutsche übersetzt** — ein bloßer Hinweis („answer in English“) änderte daran nichts.

07

Aufbau des Programms



Der letzte Pfeil ist der kritische: Alles zwischen Worker-Threads und Oberfläche läuft über Qt-Signale — direkter Zugriff hat die App reproduzierbar eingefroren.

MODUL	ZEILEN	AUFGABE
ui/main_window.py	2.732	Vier Seiten: Home, Insights, Profile, Apps
ui/desktop.py	1.655	Verdrahtung, Tray, Hotkeys, Lizenz, Updates
ui/overlay_qt.py	1.289	Die Pille: Pegel, Text, Abbrechen/Fertig/Pause

MODUL	ZEILEN	AUFGABE
ui/settings_window.py	1.219	Einstellungen, Wörterbuch, Bausteine
pipeline.py	1.057	Orchestrierung der Signalkette
textutils.py	843	Guards, Wörterbuch, Priming
usersettings.py	756	Einstellungen, atomar gespeichert
formula.py	610	Gesprochene Mathematik → LaTeX
kontext.py	340	Projekt-Gedächtnis

Qt-Fallen, die real aufgetreten sind

REFERENZZYKLUS-CRASH

Ein Lambda, das `self` fängt und im Kind-Widget liegt, baut einen Python-Zyklus. Widgets sterben dann per Garbage Collection in undefinierter Reihenfolge – sporadische Zugriffsverletzungen, deren Absturzort nie die Ursache ist.

TIMER AUF FREMDEN THREADS

`QTimer` ist auf einem Nicht-Qt-Thread wirkungslos. Der Profil-Hotkey kommt vom Tastatur-Listener – die Anzeige blieb deshalb ewig stehen, und das Halten funktionierte nie.

WIDGETS IM FALSCHEN THREAD

Der Lizenz-Dialog wurde im Listener-Thread *gebaut*, nicht nur gezeigt. Die App fror komplett ein. Beides läuft jetzt über den Signal-Bus.

ZEILENNUMMER ≠ DATENINDEX

Mit einem Suchfeld ist Zeile 0 nicht mehr Profil 0. Wer die Zeilennummer als Index nimmt, benennt stillschweigend das falsche Profil um.

08

Was gespeichert wird

Alles unter `%APPDATA%\Fleeche`, alles lokal, nichts verschlüsselt-versteckt.

DATEI	INHALT	LEBENSDAUER
<code>settings.json</code>	Hotkeys, Profile, Wörterbuch, Lizenz	dauerhaft
<code>settings.json.bak</code>	die letzte gute Fassung	bei jedem Speichern erneuert
<code>history.db</code>	Diktate für Home und Insights	abschaltbar, löschar
<code>kontext.db</code>	gelerntes Fachvokabular	90 Tage ohne Treffer
<code>fleech.log</code>	Diagnose	rollierend

Warum atomar gespeichert wird

Bis 4.9.1 kürzte das Speichern die Datei erst auf null und schrieb dann neu. Traf ein hartes Beenden genau diesen Moment, blieb eine leere Datei zurück – und das Laden fiel still auf die Vorgaben, die der nächste Speichervorgang zementierte. Weg waren Hotkeys, Profile und der Lizenzschlüssel.

Jetzt: erst vollständig in eine Nebendatei, dann `fsync`, dann `os.replace` – atomar. Es gibt nur die alte *oder* die neue Fassung, nie etwas dazwischen. Die vorherige bleibt als Sicherung liegen, aus der sich Fleech selbst heilt.

09

Lizenz und Weitergabe

Zwei Repositories: der Quellcode privat, die Installationsdateien öffentlich. Klingt widersprüchlich, ist aber der Kern der Konstruktion.

WARUM DIE INSTALLER ÖFFENTLICH SIND

Nur so kommt die Update-Prüfung ohne Zugriffstoken aus. Ein Token in einer ausgelieferten Programmdatei wäre mit `strings` in Sekunden auslesbar – und

WARUM DAS TROTZDEM TRÄGT

Wer Fleech *benutzen* darf, entscheidet ein Ed25519-signierter Schlüssel. Die App kennt nur den öffentlichen Teil – damit lässt sich prüfen, niemals unterschreiben.

gäbe Lesezugriff auf den gesamten Quellcode.

Die Sperre sitzt **vor dem Mikrofon**, nicht in der Pipeline: Ohne gültigen Schlüssel wird gar nichts erst aufgenommen. Geprüft wird offline; der Name steht mitsigniert im Schlüssel.

Ehrlich benannt: Eine lokale Prüfung lässt sich herauspatchen. Sie verhindert das Weiterreichen an Dritte, nicht das Reverse Engineering. Ein anderes Verfahren gibt es für lokal laufende Software nicht.

Updates laufen gegen das öffentliche Repository, mit SHA-256-Prüfung gegen die Release-Notizen. Passt die Summe nicht, wird die Datei verworfen. Bei Weiterleitungen wird der Authorization-Header entfernt und die Zielhost geprüft.

10

Betriebsdaten

Gemessen, nicht geschätzt — aus 1.190 Diktaten im täglichen Gebrauch.

KENNZAHL	WERT	ANMERKUNG
Diktate	1.190	über rund vier Wochen
Wörter	95.858	etwa das 3,5-Fache von „Der alte Mann und das Meer“
Sprechzeit	15,2 h	Ø 46 s je Diktat
Sprechtempo	105 WPM	etwa dreimal Tippgeschwindigkeit
Erkennung	810 ms	warm, auf der Grafikkarte
Bereinigung	4,1 s	Ø über alle Routing-Stufen
Ohne KI	3 %	trivial — direkt eingefügt

KENNZAHL	WERT	ANMERKUNG
Schnelles Modell	13 %	einfache Diktate
Volle Bereinigung	81 %	der Normalfall
Rückfall auf Rohtext	1,5 %	Guard hat gegriffen
Freihand-Dauerlast	1,2 %	eines Kerns, im Leerlauf
Projekt-Gedächtnis	0,6 ms	Abruf je Diktat

Warum gemma3:4b

An 15 echten Diktaten gegen `qwen3.5:9b` gemessen: 34 % schneller, halb so groß (3,3 GB) und dabei *wortgetreuer* – es ergänzt 0,014 statt 0,021 eigene Wörter je Diktat. Das zweite, kleine Modell entfiel: Es brachte 0,1 Sekunden und kostete Treue, während zwei Modelle dauerhaft 8,5 GB Grafikspeicher belegten.